

Méthodes et approches

Méthodes et approches

- Symboliques
- Statistiques
 - modèles probabilistes Naïve Bayes, HMM, CRF;
 - Réseaux de neurones, transformers, ...

Méthodes symboliques

Méthodes symboliques

- Elles se fondent sur la conviction que le langage peut être modélisé comme un **système structuré et gouverné par des règles**, et que l'on peut programmer ces règles pour analyser, générer ou manipuler du texte.
- Ces approches reposent sur
 - des **règles explicites** écrites par des experts
 - grammaires formelles,
 - règles morphologiques,
 - règles de désambiguïsation syntaxique ...
 - des **dictionnaires et lexiques** construits à la main
 - des ontologies et **bases de connaissances**
 - WordNet
 - des formalismes **logiques**
 - logique du premier ordre,
 - réseaux sémantiques,
 - grammaires catégorielles, etc.

Principes fondamentaux

- **Représentation symbolique :**

- chaque unité linguistique (mot, morphème, syntagme, etc.) est décrite par des symboles et des structures formelles.

- **Raisonnement déterministe :**

- le traitement s'effectue via des règles explicites, plutôt que par apprentissage probabiliste.

- **Transparence :**

- on peut expliquer pourquoi une phrase a été analysée de telle manière

- **Dépendance à l'expertise humaine :**

- la performance dépend fortement de la qualité et de l'exhaustivité des règles définies par les linguistes et ingénieurs.

Années 1950–1960 : les débuts

- Le TAL **naît** avec une approche **entièrement symbolique**.
- **Traduction automatique** par règles
- Usage de **grammaires formelles** (Noam Chomsky, 1956) pour modéliser la syntaxe
- Premiers **analyseurs morphologiques** et **parseurs syntaxiques** basés sur des règles

Années 1970–1980 : l'âge des systèmes experts

- Développement de **systèmes à base de connaissances**
 - SHRDLU pour comprendre le langage dans un monde restreint
- Utilisation de **dictionnaires électroniques** et de **bases lexicales**.
- Émergence de **grammaires unificationnelles** (Lexical Functional Grammar, Head-driven Phrase Structure Grammar).
- **Traduction automatique par règles** déployée dans des contextes réels.
 - SYSTRAN

Années 1990 : apogée et limites

- Construction de grandes **ressources symboliques** : WordNet (1995), FrameNet, etc.
- Utilisation des **ontologies et réseaux sémantiques** pour la désambiguïsation lexicale.
- Les méthodes symboliques montrent leurs limites face à la complexité et la variabilité du langage, surtout en traduction et en reconnaissance de la parole.
- Montée en puissance des **méthodes statistiques** (corpus, apprentissage automatique).

Années 2000–aujourd’hui : coexistence et hybridation

- Les approches purement symboliques reculent, mais restent utilisées dans certains domaines :
 - **correction grammaticale**,
 - analyse morphologique pour les **langues à faibles ressources**
 - **outils linguistiques spécialisés** (dictionnaires, parseurs symboliques)
- Apparition d’approches **hybrides** combinant symbolique et apprentissage statistique ou neuronal

Exemples

- niveau morphologique et lexical
 - Tokenization
 - Reconnaissance d'un mot
 - Lemmatisation
 - Étiquetage morpho-syntaxique
- niveau syntaxique
 - parsing
 - analyse syntaxique
- niveau sémantique
 - Reconnaissance d'entités nommées

Reconnaissance d'un mot ou sa génération, tokenization

Listes lexicales : Modèle de base

Forme fléchie	Base	Cat	Morphèmes
ami	ami	N	ms
amie	ami	N	fs
amies	ami	N	fp
amis	ami	N	mp
chat	chat	N	ms
chats	chat	N	mp
chatte	chat	N	fs
chattes	chat	N	fp
chien	chien	N	ms
chienne	chien	N	fs
chiennes	chien	N	fp
chiens	chien	N	mp

l'analyseur parcourt le texte et compare chaque portion de ce texte à des chaînes de caractères stockées dans le dictionnaire.

facile à implémenter

- **Problèmes ?**

Lemmatisation et génération

Listes lexicales : génération de formes fléchies

Tables des bases

Base	Cat	Modèle
ami	N	1
chat	N	2
chien	N	3

Tables des modèles

Modèle	Morphèmes	Enlever	Ajouter
1	ms	0	0
1	mp	0	s
1	fs	0	e
1	fp	0	es
2	ms	0	0
2	mp	0	s
2	fs	0	te
2	fp	0	tes
3	ms	0	0
3	mp	0	s
3	fs	0	ne
3	fp	0	nes

- une **table de base** : associe à chaque base un modèle de flexion
- une **table de modèles** : associe à chaque modèle les opérations à effectuer pour trouver la forme correspondant aux différentes formes fléchies

Tokenization

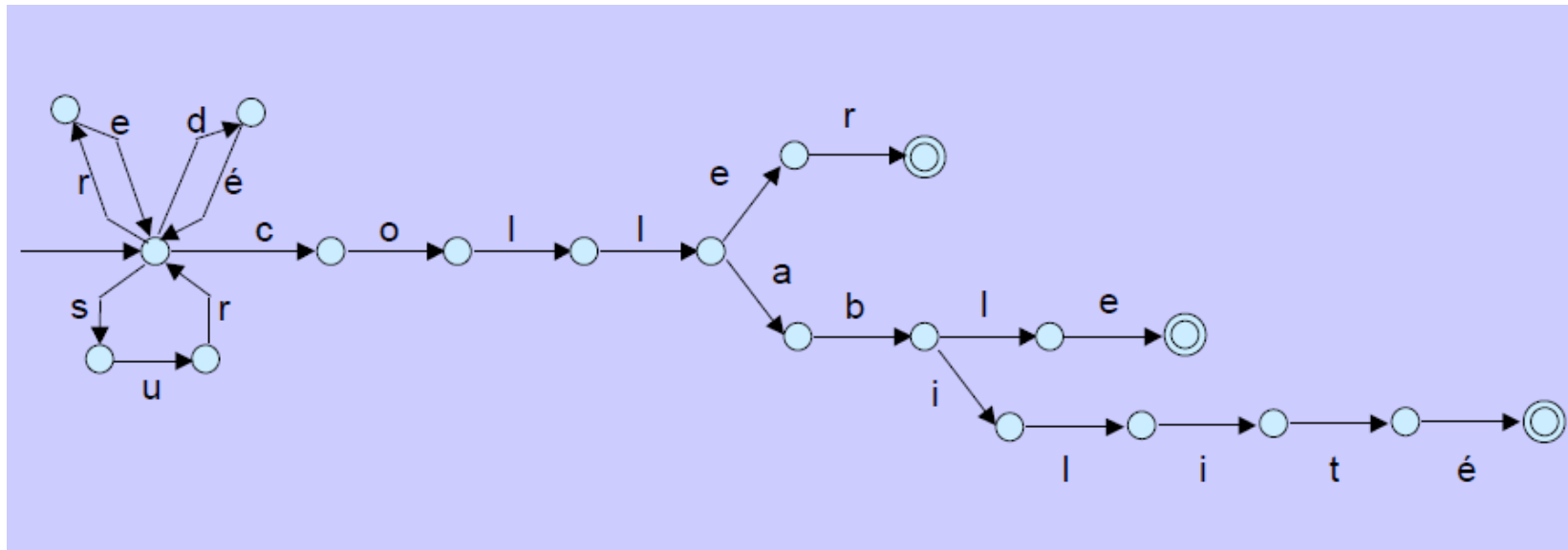
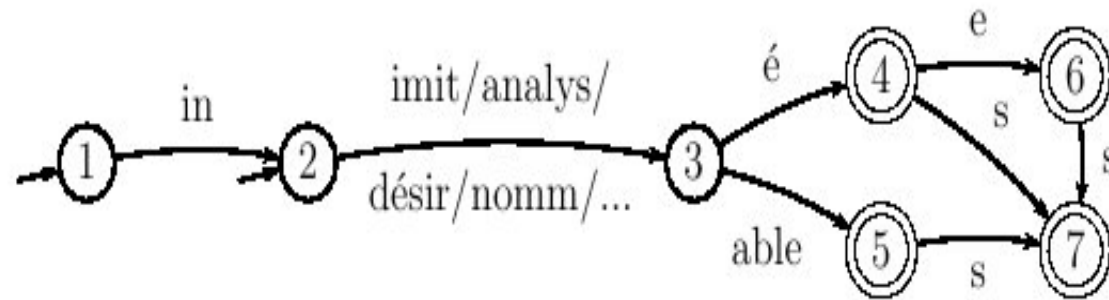
- segmenter un texte en utilisant les règles : définir des règles de découpage ou d'identification des tokens qui sont, au moins en partie, dépendantes de la langue cible
 - segmenteurs de mots
 - listes d'exception
 - règles locales

Lemmatisation et étiquetage

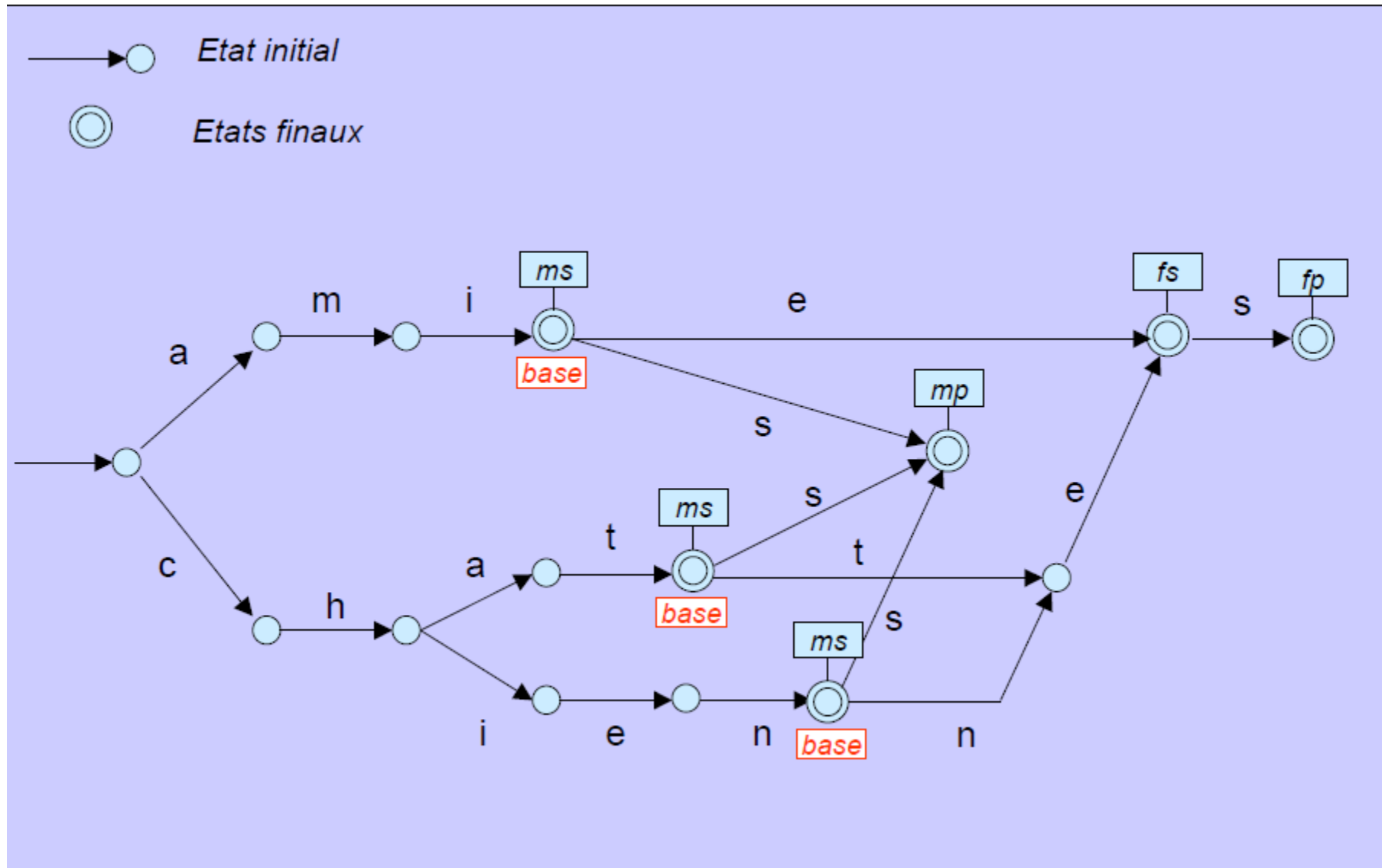
Automates finis

- Un automate fini (ou automate à états finis) est un dispositif constitué de :
 - un *vocabulaire* fini V (mots, caractères, etc.);
 - un ensemble fini Q d'*états*, parmi lesquels figurent :
 - au moins un état *initial* ;
 - au moins un état *final* ;
 - une fonction de *transition* f
- Un automate est un modèle informatique parce qu'il se traduit de façon quasi immédiate en un programme.

Automates finis : dérivation



Automates finis : étiquetage



Désambiguïsation lors de l'étiquetage

- mettre en œuvre des règles fondées sur la forme du mot :
 - si la terminaison –eur => N
 - si la terminaison –able => Adj
 - si la terminaison –ment => Adv
 - si la terminaison –er => V

Analyse syntaxique ou parsing

- **En analyse,**

- identifier les relations qui structurent les phrases d'un texte étant donnée une **grammaire de référence**
 - La grammaire a pour première fonction de fixer les règles selon lesquelles une phrase sera déclarée *grammaticale* par opposition aux phrases *agrammaticales*
 - Grammaire = l'ensemble de règles qui permettent de valider et de produire tous les énoncés d'une langue et rien qu'eux
 - Sa fonction normative
 - Définit les règles de combinaisons de mots en phrases correctes
 - Sa fonction représentative
 - Associe à une phrase sa (ou ses) représentation(s) syntaxique(s)

- **En génération,**

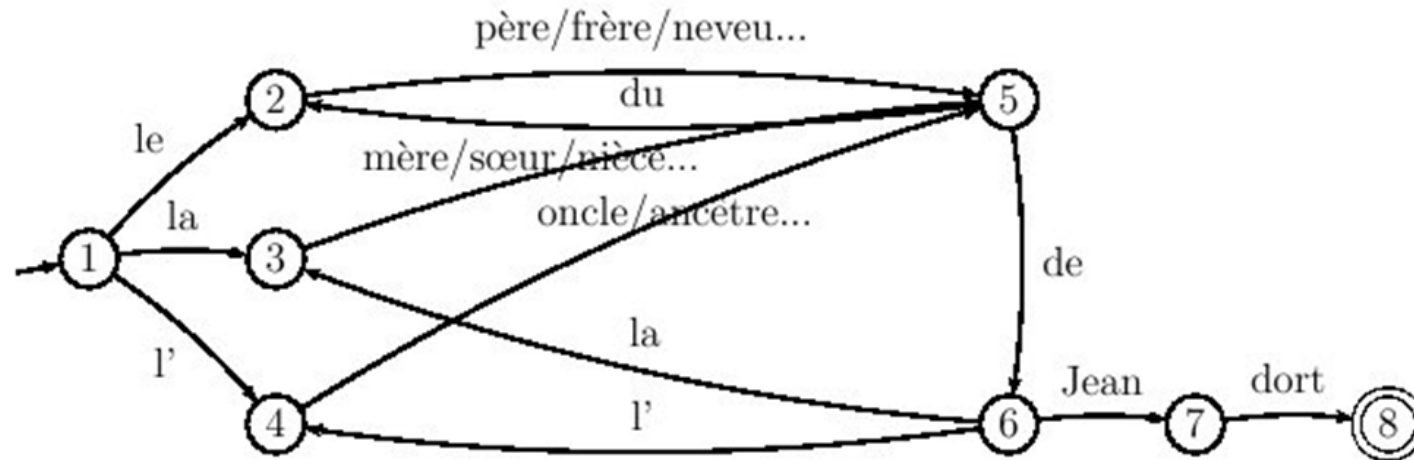
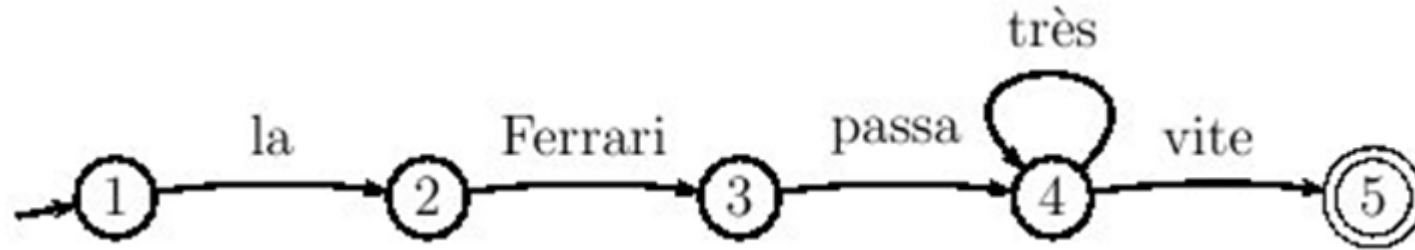
- produire des arrangements corrects de mots à partir de ces mêmes informations.

Automates finis

- Idée : Automate dont
 - le vocabulaire fini V contient l'ensemble de tous les mots possibles du français
 - chaque "chemin" correspond à une phrase syntaxiquement correcte.
- => confronté à une suite de mots quelconque, l'automate n'aurait qu'à vérifier si cette suite correspond à un de ses chemins pour savoir si elle est grammaticale

Automates finis

- un automate très simple peut "reconnaître" une infinité de chemins différents possibles => la récursivité



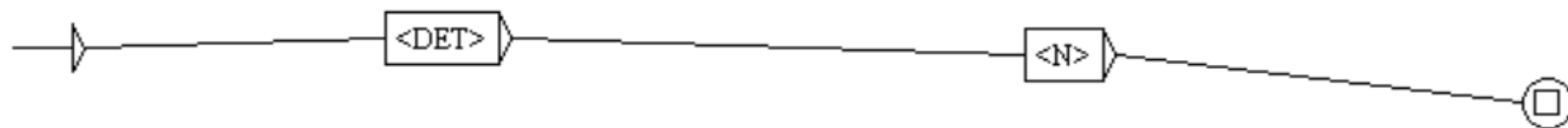
Limites des automates finis

- La grammaire de la langue française peut-elle se représenter sous la forme d'un énorme automate fini où figurerait de la récursivité ?
 - stocker en mémoire la "compétence" des locuteurs d'une langue sous la forme d'un unique automate n'est ni économique ni efficace.
 - il faudrait répéter la portion d'automate qui décrit la construction des GN, p.ex. à plusieurs endroits (sujet, objet, circonstant) dans cet automate global : *(mon voisin)GN achète (des fleurs)GN*
 - lorsqu'un enfant a entendu une seule fois un nom (ou toute autre unité lexicale, ou toute portion de structure) dans une certaine position grammaticale, il est capable de le ré-employer spontanément et instantanément dans une autre position grammaticale. : *mon voisin achète des fleurs, des fleurs sont sur la table*

⇒ transducteurs

- les automates sont incapables de produire les structures arborescentes

Graphe: GN.grf



Grammaires indépendantes du contexte

- un ensemble de règles décrivant les différents assemblages de constituants.
- un ensemble de règles de réécriture
 - Chaque règle de réécriture **associe au constituant de la grammaire indiquée en partie gauche la liste des constituants** qu'il faut avoir trouvés pour pouvoir le construire
 - Les règles **s'appliquent récursivement** jusqu'à ce que tous les mots du lexique aient été « consommés »
 - la grammaire part du symbole P (partie gauche du règle) la **remplace** partout par la partie droite correspondante
 - les nouveaux symboles qui apparaissent sont à leur tour être remplacés par l'application d'autres règles
 - une règle quelconque peut être utilisée plusieurs fois
 - Le principe de fonctionnement est la **réécriture de symboles**

Règles de réécriture

- Chaque règle possède
 - une partie gauche = le symbole à réécrire
 - une partie droite = un ou plusieurs symboles réécrivant le symbole de gauche
- S/P
 - *symbole initial*, toujours à gauche des règles
- GN, GV
 - *symboles intermédiaires (non terminaux)* à gauche et à droite des règles
- le, la, chat
 - *Symboles terminaux* (mots) à droite de la règle
- Deux types de règles
 - règles syntagmatiques, qui donnent la décomposition en constituants :
S->GN GV
GN-> DET N
 - règles terminales, qui décrivent le lexique :
N->chat
DET->le

Règles de réécriture

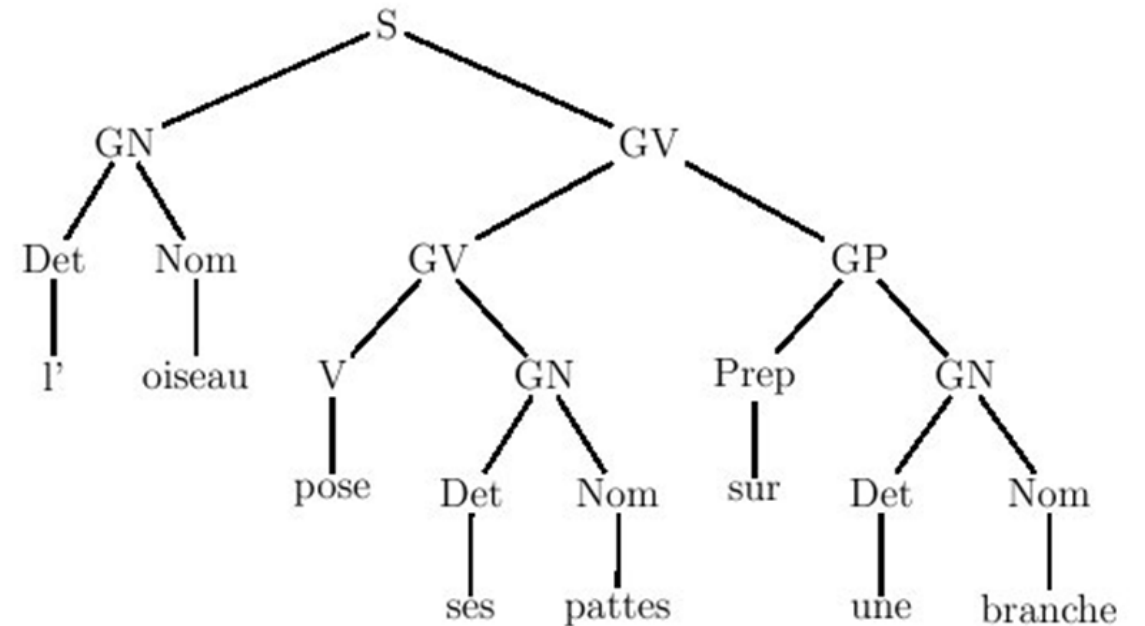
L'oiseau pose ses pates sur une branche

Règles syntagmatiques :

- S-> GN GV
- GN->DET N
- GV->GV GP
- GV->V GN
- GP->Prep GN

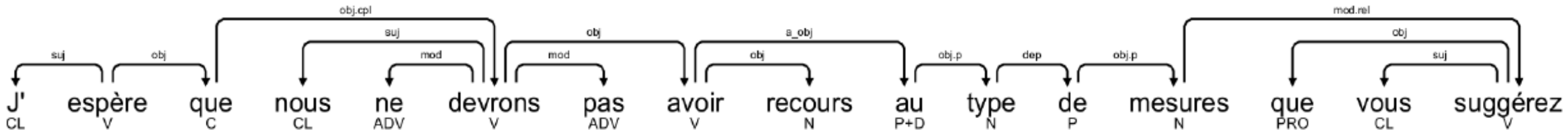
Règles terminales :

DET->l'
DET->ses
DET->une
N->oiseau
N->pates
N->brache
Prep->sur
V->pose



Grammaire de dépendance

- la structure de la phrase est représentée par un arbre
 - le verbe est le pivot de la phrase, il n'est le complément de rien
 - chaque élément est le complément d'un autre élément
 - chaque élément est ainsi *dépendant* d'un autre.



Grammaire de dépendance

- <http://alpage.inria.fr/frmgwiki/wiki/frmg-une-grammaire-du-fran%C3%A7ais>
 - Taper une phrase dans une fenêtre à gauche : « Analyser une phrase en français »
 - Lancer

Reconnaissance d'entités nommées

- Lexiques
 - listes “d’amorces” : des mots qui peuvent exprimer la catégorie recherchée :
 - *société, compagnie, filiale*, etc. => pour la reconnaissance d’entreprises
 - président, ministre, directeur, etc. => pour la reconnaissance de personnes
 - dictionnaires de noms propres
 - liste de prénoms, d’entreprises, de villes, etc.
- Les règles (patrons) spécifiques à la langue traitée)
 - amorce + de + Groupe Nominal => *société de service informatique*, etc.
 - amorce + prenom + NP => Président John Withmann
 - *amorce + de + dictionnaire de noms propres* => *hôpital d’Orléans*

Méthodes symboliques

- **Avantages :**

- Interprétables et transparents.
- Adaptés aux domaines spécialisés où les règles sont bien définies.

- **Inconvénients**

- Fortement dépendants du travail manuel d'experts.
- Peu évolutifs face à la diversité et à l'ambiguïté du langage naturel
- Trop coûteux en ressources humaines
- Non exhaustives
- Difficultés à couvrir les variations linguistiques, les erreurs, et les usages nouveaux

Méthodes statistiques

Du symbolique au statistique

- **Méthodes symboliques :**

- on écrit des règles à la main → « si un mot finit par -ment, alors c'est probablement un adverbe ».

- **Méthodes statistiques** (années 1990) : au lieu d'écrire toutes les règles, on **apprend automatiquement** les **régularités du langage** à partir de grands corpus.

=> On ne demande plus « à un expert de tout coder », mais « à la machine d'apprendre à partir des données ».

Les premiers modèles probabilistes

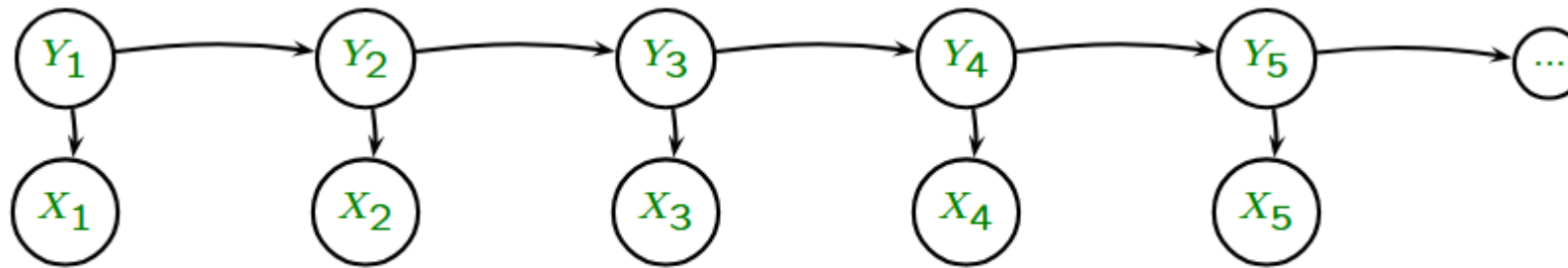
- Naïve Bayes (années 1990)
 - **Idée** : prédire une catégorie en utilisant la **probabilité** des mots.
 - Suppose que **les mots sont indépendants** (naïf, car ce n'est pas vrai en réalité).
 - la probabilité du texte = produit des probabilités de chaque mot donné la catégorie.
 - Simple, rapide, mais limité : ne tient pas compte de l'ordre des mots.

$$P(\text{catégorie} \mid \text{document}) = \frac{P(\text{document} \mid \text{catégorie}) \cdot P(\text{catégorie})}{P(\text{document})}$$

- **P(catégorie | document)** = ce qu'on cherche : probabilité que le texte ait une catégorie
 - par ex. : probabilité que le texte soit positif, négatif, etc.
- **P(document | catégorie)** = la probabilité d'observer ce texte si on connaît déjà sa catégorie.
- **P(catégorie)** = la probabilité « a priori » d'une catégorie
 - par ex. : si 70 % des critiques de ton corpus sont positives, alors $P(\text{positif})=0.7$).
- **P(document)** = probabilité globale du texte (souvent ignorée, car identique pour toutes les catégories).

Les premiers modèles probabilistes

- Modèles de Markov cachés (HMM, Hidden Markov Models)
 - Utilisés surtout pour l'**étiquetage morphosyntaxique**
 - Idée : le mot dépend du **contexte précédent**
 - *le chat mange* : *le* est suivi très probablement par un nom, *mange* est suivi d'un nom ou rien etc.
 - On modélise ça avec des **probabilités de transition** (d'une étiquette à l'autre) et d'**émission** (probabilité d'un mot donné une étiquette).
 - Premier vrai succès des méthodes statistiques en TAL

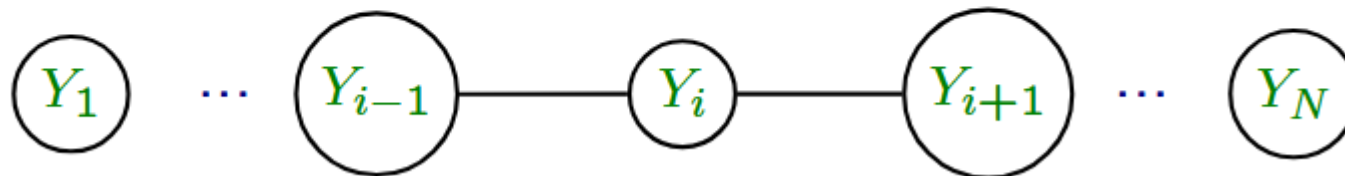


X = mot
Y = catégorie

Les premiers modèles probabilistes

- CRF, Conditional Random Fields, années 2000 (Lafferty, McCallum et Pereira 01)
 - Généralisation des HMM, plus puissants
 - Permettent de modéliser non seulement les **relations séquentielles**, mais aussi **plusieurs caractéristiques en même temps** (majuscule, suffixe, dictionnaire, etc.)
 - Possibilité d'une fonction "feature" donnée par l'utilisateur + un poids associé à chaque fonction
 - Très utilisés pour la **reconnaissance d'entités nommées**

Les CRF "linéaires"



Exemple : désambiguïsation d'un mot lors de l'étiquetage morpho-syntaxique

- livrer toutes les étiquettes possibles au module de désambiguïsation
- associer à ce mot l'étiquette la plus probable (l'analyseur choisit parmi toutes les catégories possibles d'un mot celle qui est la plus plausible dans un contexte immédiat)
 - la probabilité des séquences d'étiquettes est en général « apprise » sur un corpus déjà étiqueté
 - La probabilité que l'étiquette B succède à l'étiquette A est estimée par la fréquence du couple AB dans le corpus de référence divisée par le produit des fréquences de chaque étiquette
- Prédire une étiquette selon le modèle (le corpus de référence)

Les réseaux de neurones (2010–)

- Réseaux neuronaux classiques (feed-forward, RNN, LSTM)
 - On représente les mots par des **vecteurs numériques** (word embeddings comme Word2Vec, GloVe)
 - Les réseaux apprennent à capter les **relations contextuelles**.
 - LSTM (Long Short-Term Memory) : capables de traiter des **séquences longues**, par ex. phrases entières.
 - Applications : traduction automatique neuronale (Google Translate dès 2016), analyse de sentiment, etc.

Transformers (2017-) : “Attention is All You Need” (Vaswani et al., 2017)

- basés sur le **mécanisme d’attention** : le modèle « regarde » tous les mots d’une phrase en même temps et pèse leur importance.
- meilleurs que les réseaux de neurones
- entraînement parallèle plus rapide
- meilleure gestion des contextes longs
- Exemples de modèles : BERT, GPT, T5
- Aujourd’hui, les **transformers** sont la base de presque tout le TAL moderne (traduction, résumé, chatbots, recherche d’information, etc.

Exemple de l'étiquetage morpho-syntaxique d'une phrase

La maison est belle

- **Embeddings** : Chaque mot est transformé en un **vecteur numérique**

La → [0.2, -0.1, 0.5]

maison → [0.9, 0.3, -0.4]

est → [0.1, -0.8, 0.7]

belle → [0.6, 0.5, 0.2]

#les chiffres sont les coordonnées d'un vecteur dans un **espace à 3 dimensions appris pendant l'entraînement**

- Passage dans un **réseau de neurones** (BiLSTM) ou Transformer : Chaque mot reçoit un **vecteur enrichi par son contexte**.

Contexte("maison") → [1.2, 0.4, -0.7, 0.3]

Ce vecteur est enrichi par embedding de **position** (pour savoir à quelle place il est) et les couches d'attention qui modifient la représentation en fonction du **contexte**.

maison aura une représentation différente selon la phrase dans *La maison est belle* et *J'habite dans cette maison*

- **Prédictions** : Pour chaque mot, le réseau produit des probabilités

La → {DET: 0.95, NOM: 0.02, ADJ: 0.03} → DET

maison → {NOM: 0.85, ADJ: 0.10, VERBE: 0.05} → NOM

est → {VERBE: 0.90, AUX: 0.08, NOM: 0.02} → VERBE

belle → {ADJ: 0.92, NOM: 0.05, VERBE: 0.03} → ADJ

#un réseau de neurones transforme un vecteur de mot en **probabilités sur les étiquettes**

- **Résultat** :

La/DET maison/NOM est/VERBE belle/ADJ

Algorithmes d'apprentissage

- Supervisé
- Non supervisé
- **semi-supervisé**
- **par renforcement**
- **par transfert**
- **multi-tâches**

Apprentissage supervisé

- Définition
 - on apprend à partir d'exemples annotés
 - donner aux machines la capacité d'« apprendre » à partir de données via des modèles statistiques
 - les informations pertinentes sont tirées d'un ensemble de données d'entraînement.
- Pourquoi utiliser l'apprentissage supervisé en TAL ?
- Applications :
 - classification de texte, reconnaissance d'entités nommées, analyse de sentiments, etc.

Tâches typiques de l'apprentissage supervisé

- **Classification**

- Prédire une catégorie pour chaque exemple d'entrée.
- L'ensemble des sorties possibles est fini et défini à l'avance (classes).

Tâches typiques de l'apprentissage supervisé

- **Régression**

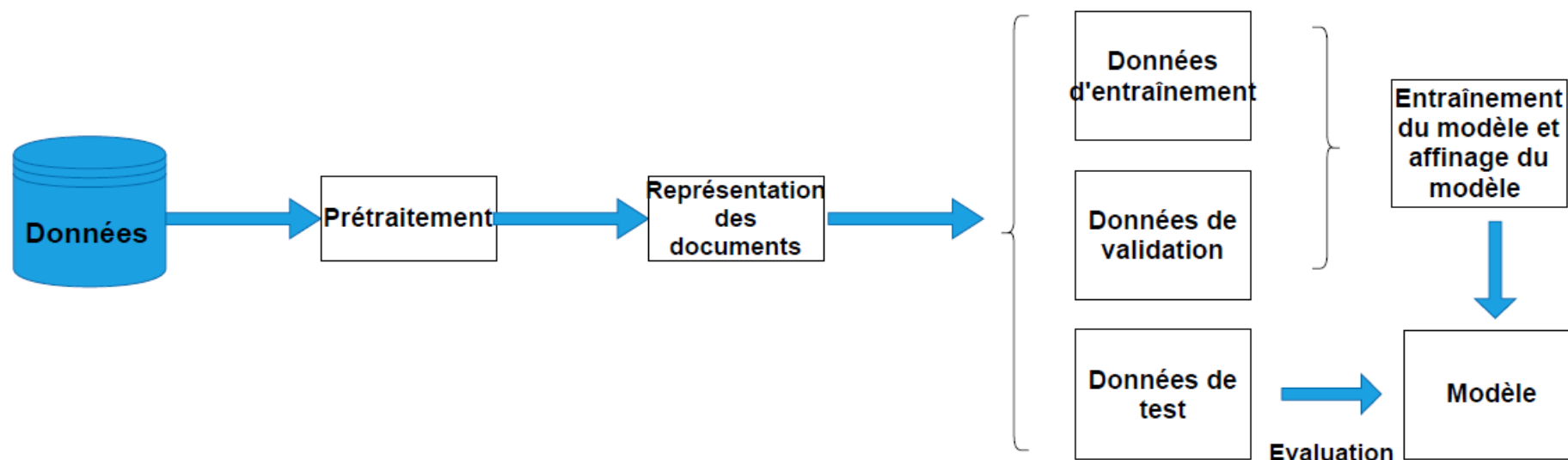
- Prédire une valeur continue à partir des données d'entrée.
- La sortie est une variable numérique continue (réelle).

Tâches typiques de l'apprentissage supervisé

- **Séquence étiquetée** (aussi appelée « étiquetage de séquence »)
 - Associer une étiquette à chaque élément d'une séquence d'entrée.
 - Exemples :
 - Analyse morpho-syntaxique
 - Reconnaissance d'entités nommées (NER)
 - Segmentation

#	Forme	Lemme	Catégorie
1	Scarlett	Scarlett	NAM
2	joue	jouer	VER:pres
3	avec	avec	PRP
4	le	le	DET:ART
5	chat	chat	NOM
6	.	.	PUNCT

Approches par apprentissage



1. une phase de **prétraitement** qui consiste à supprimer les mots vides, la ponctuation, les caractères spéciaux, à lemmatiser les mots, etc ;
2. une phase de **représentation des documents** ou de *feature engineering* qui consiste à transformer les textes en des valeurs numériques ;
3. une phase d'**entraînement et construction du modèle** qui consiste à entraîner et affiner différents modèles à partir d'un corpus d'apprentissage et de validation ;
4. une phase d'**évaluation du modèle** qui consiste à évaluer les performances du modèle sélectionné à l'étape précédente sur un nouveau jeu de données (test).

Apprentissage non supervisé

- on donne à l'algorithme seulement des données brutes (textes, phrases, mots) non annotées et il doit découvrir tout seul des structures cachées.
 - des regroupements ou des relations entre mots
 - etc.
- un algorithme apprend à identifier des structures ou des modèles dans un jeu de données sans avoir de sorties pré-établies.
- L'objectif est de permettre à l'algorithme de découvrir les relations et les structures cachées dans les données par lui-même.

Apprentissage non supervisé : techniques

- **Clustering (regroupement)** : mettre ensemble des textes ou mots qui se ressemblent, regrouper un ensemble d'objets de manière à ce que les objets dans le même groupe (ou cluster) soient plus similaires entre eux qu'avec ceux d'autres groupes
- **Modèles de thématiques (topic modeling)** : découvrir les thèmes cachés dans un grand corpus de documents.
- **Word embeddings et représentations non supervisées** : apprendre les relations entre mots à partir du contexte
- **Modèles neuronaux auto-supervisés** : apprendre à prédire une partie manquante du texte à partir du reste.
- **Réduction de dimensionnalité** : Simplifier les données en réduisant le nombre de variables à considérer, tout en conservant le plus possible l'information essentielle

Apprentissage non supervisé : avantages

- Moins coûteux que l'apprentissage supervisé qui demande l'annotation préalable de données
- permet d'explorer de grandes masses de texte, d'en extraire des régularités et d'aider ensuite les méthodes supervisées.
 - Possible à être utilisé en amont de l'apprentissage supervisé

Apprentissage semi-supervisé

- une méthode qui se situe entre l'apprentissage supervisé et non supervisé
- **utilise à la fois des données étiquetées** (où les sorties sont connues) **et des données non étiquetées** (où les sorties ne sont pas connues) pour former un algorithme.
- Utile lorsque l'obtention de données étiquetées est coûteuse ou laborieuse, mais où l'on dispose d'une grande quantité de données non étiquetées.

Apprentissage semi-supervisé : techniques

- **Propagation de label :**

- Cette méthode **exploite un petit nombre de données étiquetées pour en déduire les étiquettes des données non étiquetées.**
- repose sur l'hypothèse que les données similaires auront tendance à partager la même étiquette.
- => les étiquettes des exemples connus sont progressivement étendues aux exemples proches, ce qui permet d'apprendre efficacement même avec peu de données annotées

- **Apprentissage par co-formation :**

- on suppose que **chaque instance de données peut être décrite par deux ensembles de caractéristiques différentes**, appelés "vues", qui fournissent des informations complémentaires, lorsqu'il y a deux vues différentes mais complémentaires des données,
 - Ex : classer des pages web : une vue basée sur le texte contenu dans la page et l'autre sur les liens hypertextes pointant vers cette page.

Apprentissage par renforcement

- une méthode d'apprentissage automatique où un **agent apprend à prendre des décisions en interagissant avec un environnement**.
- À chaque action, **l'agent reçoit des récompenses ou des pénalités**, qui l'aident à évaluer la qualité de ses décisions.
- Le but est d'optimiser les actions de l'agent afin de maximiser la somme totale des récompenses obtenues au fil du temps.

Apprentissage par transfert

- une technique d'apprentissage automatique qui implique **le transfert de connaissances acquises dans un contexte à un autre**.
 - un fine-tuning : l'on conserve la majorité des paramètres appris sur la tâche de départ et l'on ré-entraîne le modèle afin de l'adapter aux spécificités de la nouvelle tâche.
- utile pour les domaines où les données sont rares ou coûteuses à obtenir, car elle permet de construire des modèles performants sans nécessiter de vastes ensembles de données spécifiques à chaque problème.

Apprentissage multi-tâches

- Permet à un modèle unique d'**apprendre simultanément plusieurs tâches connexes**.
- Le modèle utilise des représentations communes pour les différentes tâches, ce qui permet non seulement de gagner en efficacité en termes de temps et de ressources computationnelles, mais aussi d'améliorer la généralisation de chaque tâche individuelle grâce aux connaissances acquises à partir des autres tâches.
- Exemples : REN + Analyse de sentiments = **entraîner un modèle de NLP sur ces deux tâches simultanément**
 - *Apple a lancé un nouveau produit à New York*
=> comprendre que *Apple* est une organisation peut aider le modèle à mieux interpréter les sentiments liés à cette entité dans différents contextes.

Modèles d'apprentissage supervisé

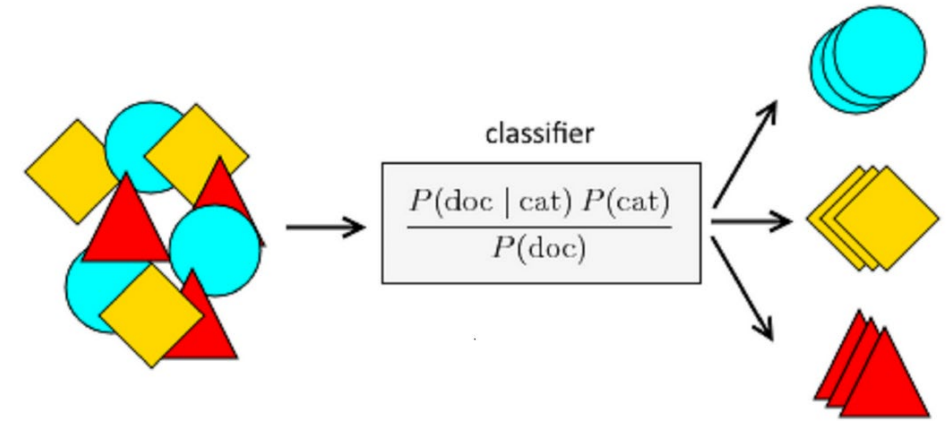
- Approche de surface
 - Naïve Bayes
 - Régression logistique
 - SVM
 - Arbres de décision / Random Forests
- Approche profonde

Modèle de l'approche de surface

- **Régression logistique** : combine linéairement les mots ou vecteurs, puis donne une probabilité d'appartenir à une classe (*spam* ou *non-spam*).
- **Naive Bayes** : algorithme probabiliste
 - calcule la probabilité qu'un mot apparaisse dans un *spam*
- **Arbre de décision** : non-linéaires, construit des règles du type :
- **Random Forest** : fonctionnent **en parallèles**
 - extraient aléatoirement des sous-échantillons différents du jeu de données initial, entraînent plusieurs modèles en parallèle sur ces échantillons combinent leurs prédictions (par ex. vote majoritaire)
- **AdaBoost, XGBoost** : fonctionnement séquentiel
 - les différents classifieurs sont entraînés séquentiellement, chacun modifiant la prédiction effectuée par le classifieur précédent ce qui “booste” la performance
=> Les observations mal classées prennent plus d'importance au fur et à mesure

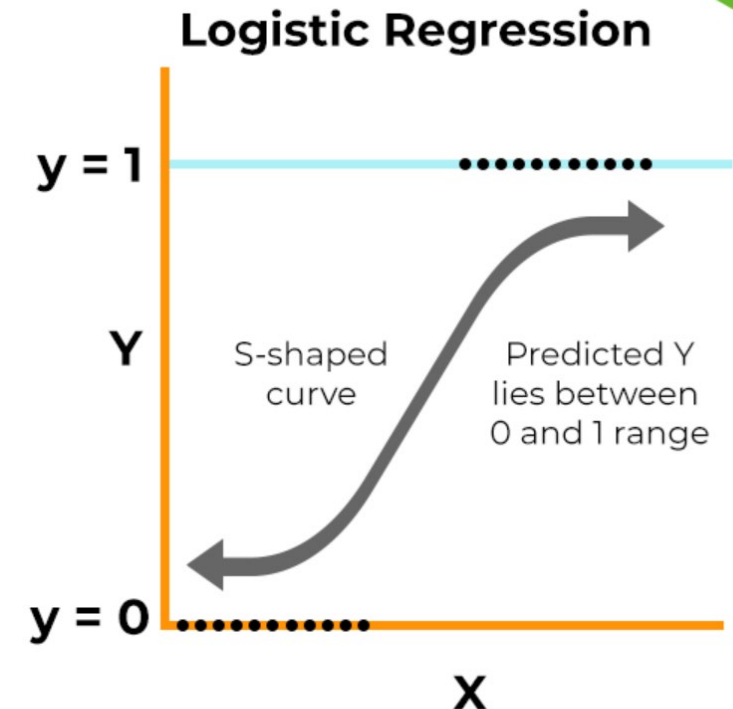
Naive Bayes

- Classe un texte dans une **catégorie** donnée, à partir des **mots** qu'il contient.
- Naive Bayes suppose que **les mots sont indépendants entre eux**, d'où le nom "naïf"



Logistic Regression

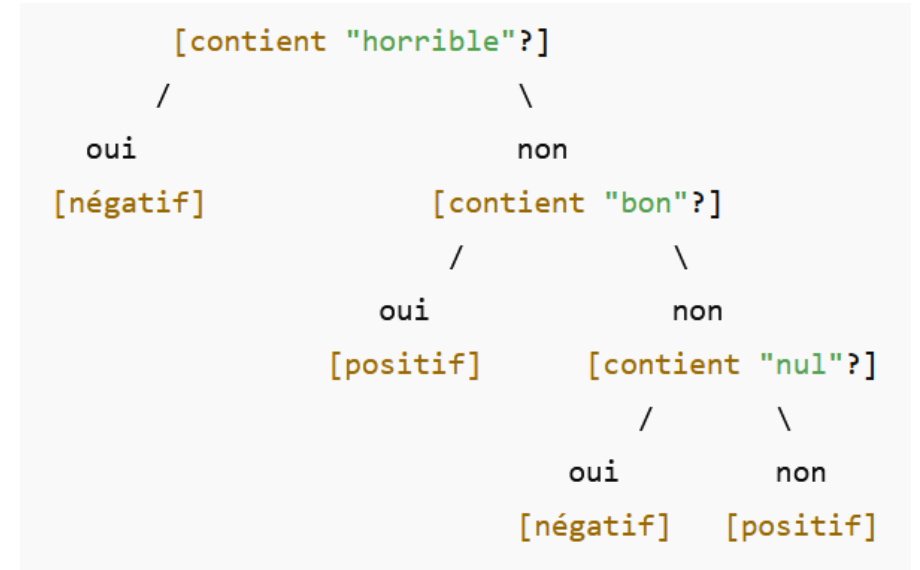
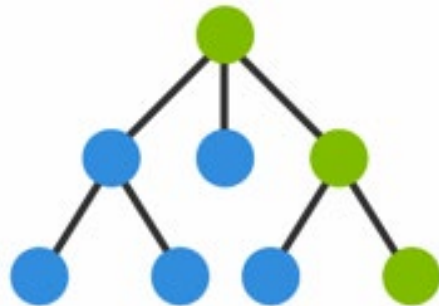
- attribue une probabilité à chaque classe
- un **modèle de classification binaire ou multi-classe**
- la **prédiction** est ensuite **effectuée selon un seuil fixé**
 - Pour une classification binaire par exemple, si la probabilité obtenue est **supérieure à 0,5**, la **classe est prédite comme positive**,
 - en revanche, si la probabilité est **inférieure à 0,5**, la **classe est prédite comme négative**.



Logistic Regression – Sigmoid Function

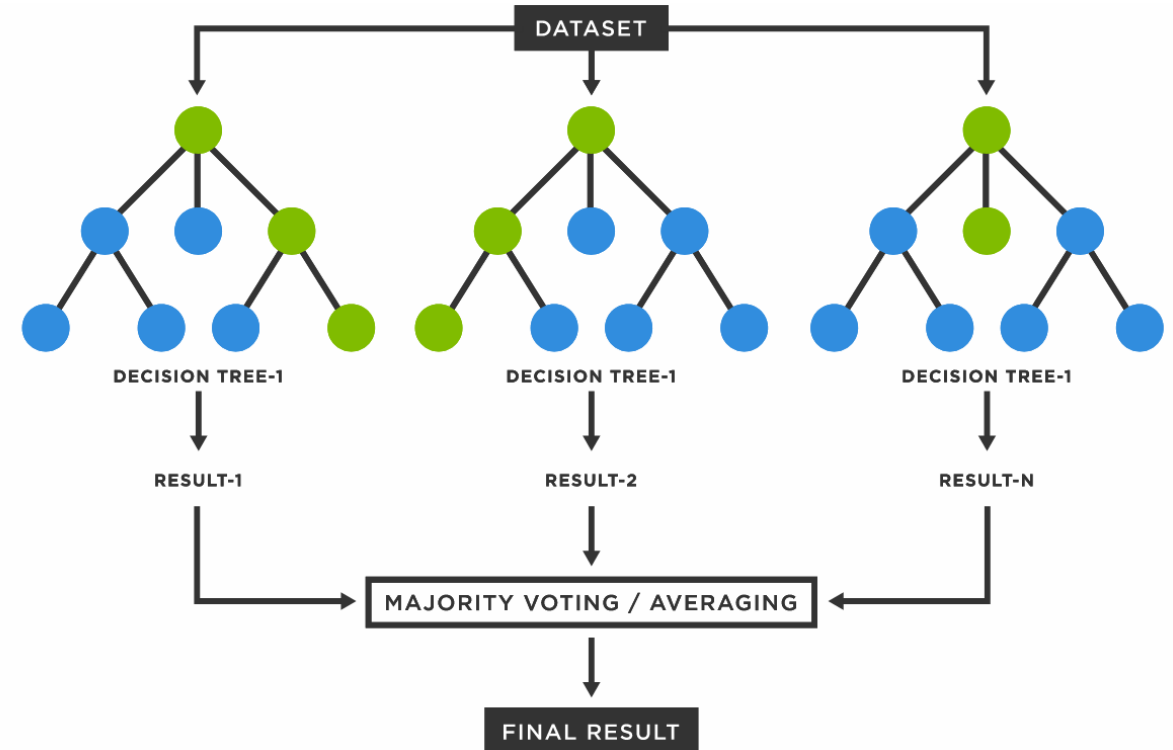
Arbre de décision

- un **modèle prédictif** qui fonctionne un peu comme une série de **questions oui/non** organisées de manière hiérarchique pour prendre une décision.
- Chaque nœud représente une question (sur une caractéristique des données).
- Chaque branche correspond à une réponse possible (souvent "oui" ou "non").
- Chaque feuille donne une prédiction (la classe ou la valeur).
- le modèle considère comme validées, uniquement les données qui ont servi à faire l'apprentissage, ne reconnaissant aucune autre donnée qui soit un peu différente de la base initiale. => Cette faiblesse a conduit à la création des algorithmes de type **Random Forest, composés d'un ensemble d'arbres de décision.**



Random Forest/Arbres de décision

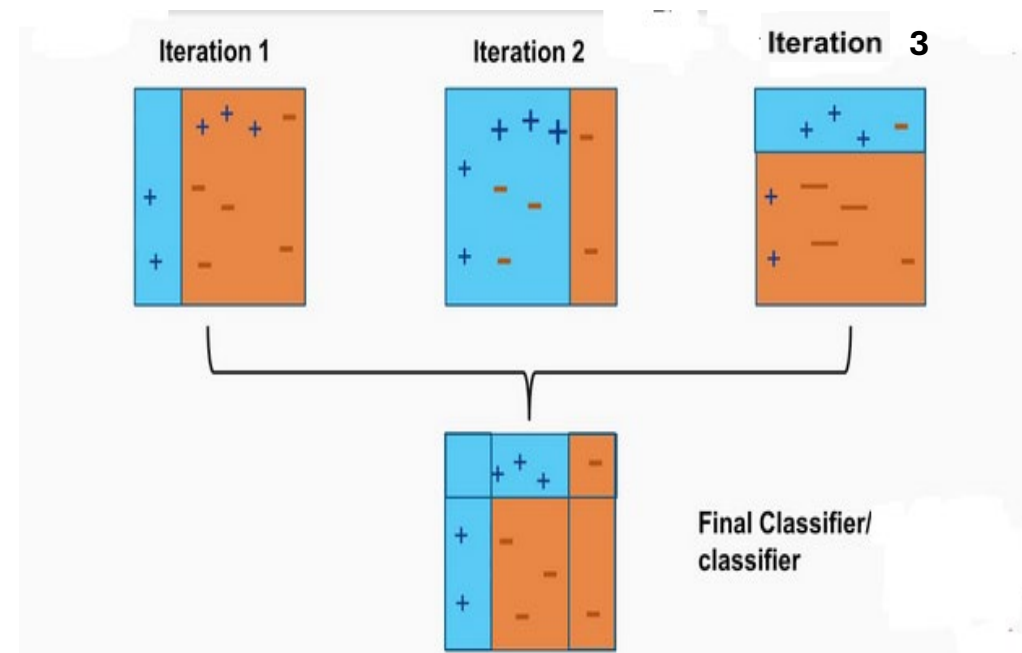
- combine les résultats de plusieurs modèles faibles (arbres) pour en faire un modèle plus fort et plus robuste.
- **composés d'un ensemble d'arbres de décision,**
 - plusieurs arbres de décision sont appliqués sur des sous-ensembles aléatoires de données,
 - Les **résultats** étant ensuite **moyennés**
- Pour une nouvelle donnée, chaque arbre fait une prédiction, et la majorité (vote) détermine la sortie finale.
- Fonctionne pour les tâches de **classification** et de **régression**



AdaBoost (Adaptive Boosting)

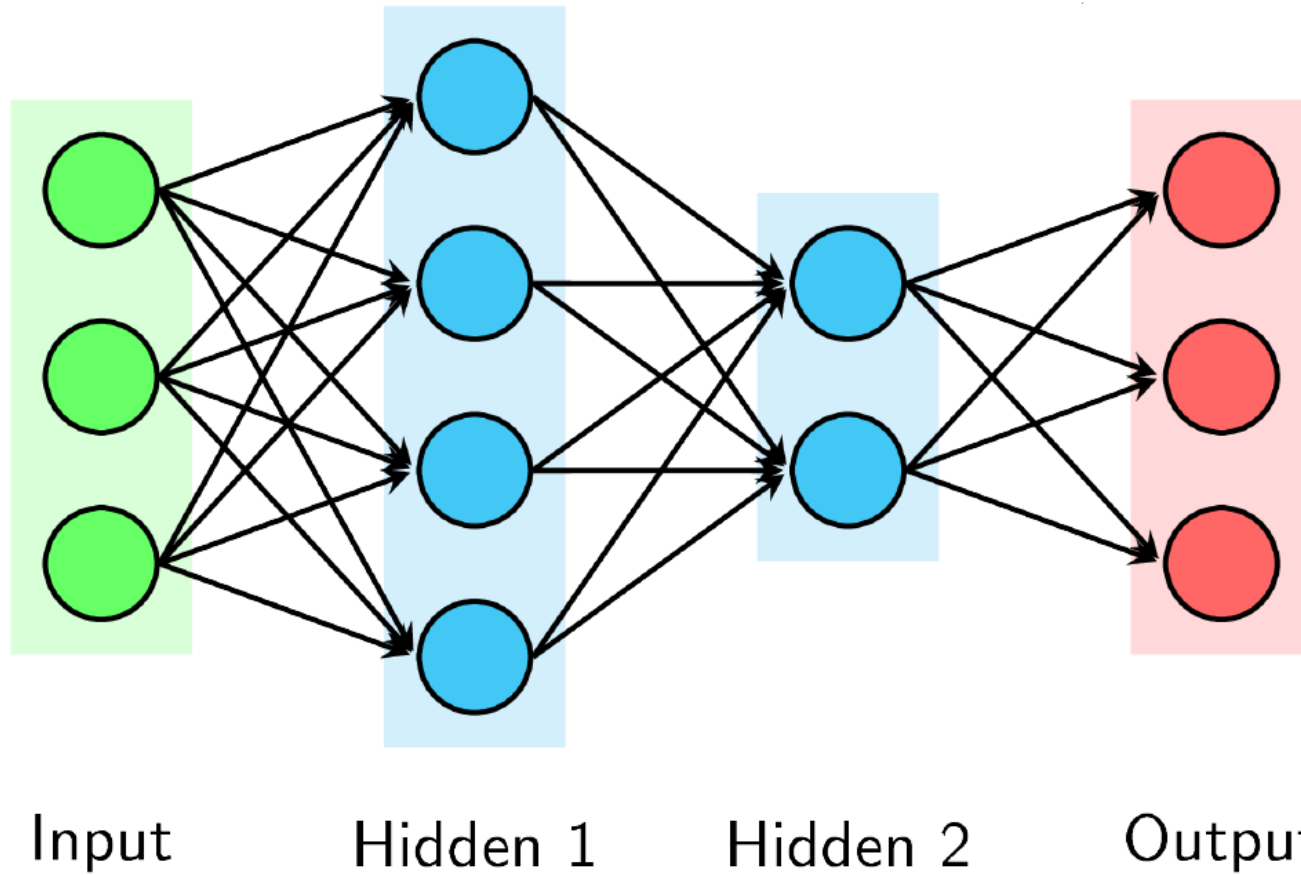
Combine plusieurs modèles faibles (souvent des arbres de décision très simples) pour créer un modèle fort.

- Tous les exemples ont le même poids.
- Après avoir évalué le premier arbre, le classifieur est modifié en donnant plus de poids aux observations qui sont difficiles à prédire, de façon à se concentrer plus sur les erreurs.
- Chaque nouveau modèle est entraîné pour corriger les erreurs du précédent.
- On apprend un deuxième modèle, puis un troisième, etc.
- Combinaison finale : on fait une moyenne pondérée des prédictions des modèles faibles.



Apprentissage profond

- Le concept clé de l'apprentissage profond réside dans les **réseaux de neurones**.
- Ces réseaux sont composés de plusieurs **neurones connectés** entre eux, formant un modèle qui prend des décisions de manière **comparable au cerveau humain**.
- D'un point de vue mathématique, un réseau de neurones est constitué de couches de neurones où **chaque neurone effectue des opérations mathématiques sur ses entrées**.
- Les neurones sont organisés en couches :
 - les **couches d'entrée** reçoivent les données initiales,
 - les **couches cachées** traitent les informations de manière non-linéaire
 - les **couches de sortie** produisent le résultat final.
- La sortie d'une couche devient l'entrée de la couche suivante.



un modèle ayant **une profondeur 2**, c'est-à-dire ayant **2 couches cachées**.

Fonction d'activation

- un élément clé d'un réseau de neurones
- décide comment un neurone « réagit » à l'information qu'il reçoit.
 - Chaque neurone d'un réseau reçoit plusieurs entrées (des nombres), les combine, puis doit produire une sortie.
 - Sans fonction d'activation, le neurone ferait juste un calcul linéaire (comme une ligne droite sur un graphique)
 - La fonction d'activation ajoute une non-linéarité
- une décision que prend le neurone :
 - Est-ce que le signal est assez fort pour « s'activer » ? Si oui, il transmet quelque chose, sinon il reste silencieux (ou transmet peu).

Fonctionnement de base d'un réseau de neurones

- **Propagation avant :**

- Les données d'entrée passent à travers le réseau couche par couche.
- Si X est le vecteur d'entrée, la sortie de la première couche cachée peut être calculée comme : $h_1 = f(W_1X + b_1)$

- **Fonction de perte** (mesure de l'erreur du réseau) :

- Une fonction de perte mesure l'écart entre la sortie prédite par le réseau et la valeur réelle attendue.
- Le but de l'entraînement est de minimiser cette perte.

- **Rétropropagation :**

- Pour minimiser la fonction de perte, on utilise la rétropropagation pour calculer le gradient de la perte par rapport aux poids et biais du réseau.
- Pour apprendre, le réseau calcule combien chaque poids contribue à l'erreur (grâce à la rétropropagation), puis ajuste ces poids dans la direction qui réduit cette erreur.

RNN

- Les **réseaux de neurones récurrents** (RNNs) sont des réseaux de neurones qui traitent les éléments d'une séquence dans l'ordre, c'est-à-dire, **séquentiellement**.
- Ils sont particulièrement utiles pour
 - la reconnaissance automatique de l'écriture manuscrite,
 - la reconnaissance de la parole
 - la traduction automatique.

Transformers

- Les transformers ont été développés pour **pallier les limitations des architectures de réseaux de neurones récurrents (RNN)**
 - Les RNN, **traitent les données séquentiellement**, ce qui ralentit le traitement.
 - Les RNN **difficultés avec les dépendances longue distance**
- Les transformers utilisent un **mécanisme d'attention** globale qui permet au modèle de **regarder toute la séquence d'entrée simultanément**, améliorant ainsi la vitesse du traitement et la prise en compte des dépendances longue distance.
- Cette **approche parallèle** a non seulement conduit à des gains substantiels en efficacité et en performance, mais a également ouvert la voie à l'entraînement de modèles très grands et puissants, tels que GPT et BERT

RNN vs Transformers : mécanisme d'attention

- Mécanisme d'attention

- concept général ou le processus complet par lequel un réseau de neurones (souvent dans le traitement du langage ou de la vision) apprend à **se concentrer sur certaines parties pertinentes des entrées** lorsqu'il produit une sortie.
 - Le mécanisme d'attention n'est pas exclusif aux Transformers — il a été introduit d'abord dans les RNN
 - Le mécanisme d'attention **permet au décodeur de regarder les états cachés de l'encodeur** (tous les mots sources) au lieu de dépendre uniquement du dernier état caché.
 - Les Transformers ont ensuite remplacé complètement les RNN en s'appuyant **uniquement sur le mécanisme d'attention**.
 - Le Transformer introduit la **Self-Attention** (l'attention sur soi-même), où chaque position d'une séquence regarde toutes les autres positions.
 - L'attention devient **le cœur** du modèle : c'est le seul moyen de propagation de l'information entre les tokens.
- Dans un **RNN** => l'attention est un **module additionnel**.
- Dans un **Transformer** => l'attention est **le mécanisme fondamental**

Bert

le modèle statistique BERT (Bidirectional Encoder Representations from Transformers),

- repose sur la structure de modèle neuronal **Transformers**
- a été conçu pour apprendre des **représentations bidirectionnelles** profondes **à partir de textes non étiquetés** en prenant obligatoirement en compte les **contextes de gauche et de droite conjointement**, et ce, dans toutes les couches du Transformer.
- **peut être réentraîné/affiné** (en anglais : fine-tuned) sur un nouveau jeu de données, le spécialisant sur une tâche en particulier.
- BERT peut être donc **utilisé**
 - **tel quel** en y ajoutant une couche pour résoudre la tâche spécifique
 - on peut extraire ses plongements pour les **utiliser dans un tout autre modèle**.

Plongements lexicaux basés sur les transformeurs

- Contrairement aux modèles antérieurs tels que Word2Vec ou GloVe qui génèrent un vecteur statique pour chaque mot, les transformeurs produisent des **représentations dynamiques** où **le vecteur associé à un mot peut varier selon le contexte dans lequel il apparaît**.
- BERT se distingue par son **approche bidirectionnelle**, analysant chaque mot dans le contexte des mots qui le précèdent et le suivent dans une phrase.
- Cela est réalisé par un **mécanisme d'attention** qui **pèse l'importance relative des autres mots** pour chaque prédiction de mot.

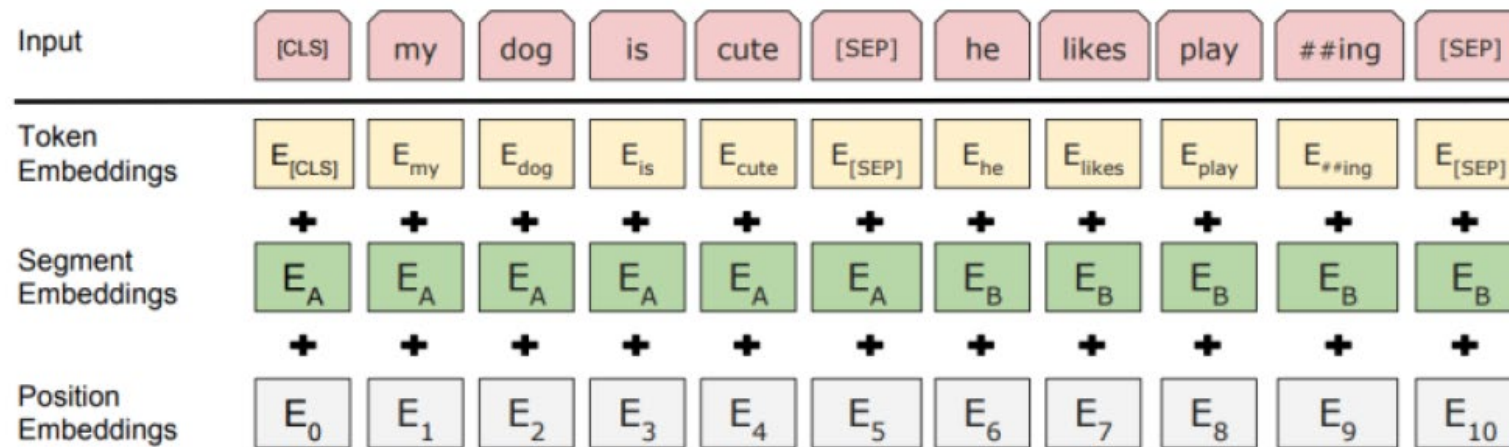
BERT : plongements

Le plongement d'entrée est réalisé en **plusieurs étapes**

- **Les textes sont prétraités en insérant des symboles spéciaux** dans le texte brut, pour indiquer au modèle certaines informations sur les mots et sur la composition du texte en entrée (début de séquence, séparation entre plusieurs séquences, etc.).
- **Les mots sont décomposés en sous-mots** qui sont représentés par leur identifiant dans ce vocabulaire.
 - La décomposition en sous-mots est faite **automatiquement par un algorithme** appelé **WordPiece** (dans BERT)

BERT : plongements

- Chaque élément est associé
 - au **token embedding** (plongement d'un token).
 - au **position embedding** (plongement de position) : l'information structurelle de la phrase
 - au **segmentation embedding** (plongement de segmentation de phrase) : l'information de positionnement des phrases dans l'entrée.
- La **représentation finale de chaque sous-mot** en entrée est la **somme de ces trois vecteurs** de représentation qui lui sont associés.



Bert

- BERT produit des représentations (plongements) contextualisées des mots du vocabulaire issus de corpus non labellisés selon **deux tâches** auto supervisées simultanément :
 - la modélisation du langage masqué
 - la prédiction de la phrase suivante.

Tache 1 : Modélisation du langage masqué (MLM)

- Bert **prédit le mot voisin à partir de son entourage de manière bidirectionnelle.**
- BERT **prend en compte simultanément les contextes droit et gauche du mot cible de la séquence d'entrée**, ce qui permet au modèle
 - de **capturer le contexte d'un mot** par rapport à d'autres mots **dans les deux sens**, gauche et droite,
 - de **produire des représentations contextualisées**
- Pendant la tache de modélisation du langage masqué,
 - **15% des tokens** de chaque séquence d'entrée du réseau **est masqué de façon aléatoire.**
 - **Ces mots sont supprimés** dans la séquence et **remplacés par le token <MASK>.**
 - Après cette étape, le modèle essaie de **prédire ces mots en regardant les mots non masqués** appartenant à la séquence.

Tache 2 : Prédiction de la phrase suivante

- Le modèle **reçoit des paires de phrases en entrée et apprend à prédire si la deuxième phrase de la paire est la phrase suivante** dans le document original.
- Pendant l'apprentissage,
 - 50% des entrées sont une paire dans laquelle la deuxième phrase est la phrase suivante dans le document original et labellisée *Is-Next*,
 - dans les 50% restants, une phrase aléatoire du corpus est choisie comme deuxième phrase et labellisée (NotNext).

Bert

- **entraîné sur de très larges quantités de données non annotées** afin de bâtir des représentations linguistiques suffisamment robustes.
- a été réalisé en s'appuyant sur un corpus de 3,3 milliards de mots provenant de BooksCorpus (800 Millions de mots) et du Wikipedia Anglais (2.5 Milliards de mots)
- s'agissant de Wikipedia, seuls les passages de texte ont été extraits. Les listes, tables et titres n'ont pas été pris en compte.

Variantes de BERT

- **BERT-base** est construit sur la base Wikipedia (2,5 milliards de mots) et BookCorpus (800 millions de mots)
- **BioBERT** pour le domaine biomedical
- **SciBERT** pour les corpus de publications scientifiques
- **mBERT-Base**, une variante multilingue, a été pré-entraîné sur des données multilingues (104 langues [21](#)) issues de Wikipedia.
- Le vocabulaire diffère d'une version de BERT à une autre

- **FlauBERT**

- Entraîné sur 71 gigaoctets de textes composés de Wikipédia, des articles du Monde, des ouvrages francophones du projet Gutenberg (dont Flaubert), des débats du Parlement européen
 - Hang Le, Loïc Vial, Jibril Frej, Vincent Segonne, Maximin Coavoux, Benjamin Lecouteux, Alexandre Allauzen, Benoît Crabbé, Laurent Besacier, Didier Schwab. **FlauBERT: Unsupervised Language Model Pre-training for French. 2019**
- **FlauBERT a été entraîné** uniquement sur une seule tâche : la **modélisation du langage masqué (MLM)**

- **CamemBERT**

- Entraîné sur la section française du corpus multilingue OSCAR (collaboration entre INRIA et Facebook)
- Choisit des mots à prédire de manière dynamique

Comparaison des modèles : RoBERTa, CamemBERT et FlauBERT

	<i>RoBERTa</i> _{BASE}	<i>CamemBERT</i> _{BASE}	<i>FlauBERT</i> _{BASE} / <i>FlauBERT</i> _{LARGE}
Langue	Anglais	Français	Français
Données d'apprentissage	160 GB	138 GB	71 GB
Objectifs de pré-entraînement	MLM	MLM	MLM
Nombre total de paramètres	125 M	110 M	138M/373M
Tokenisation	BPE 50K	SentencePiece 32K	BPE 50K
Masque	Dynamique + sous-mots	Dynamique + mot entier	Dynamique + sous-mots

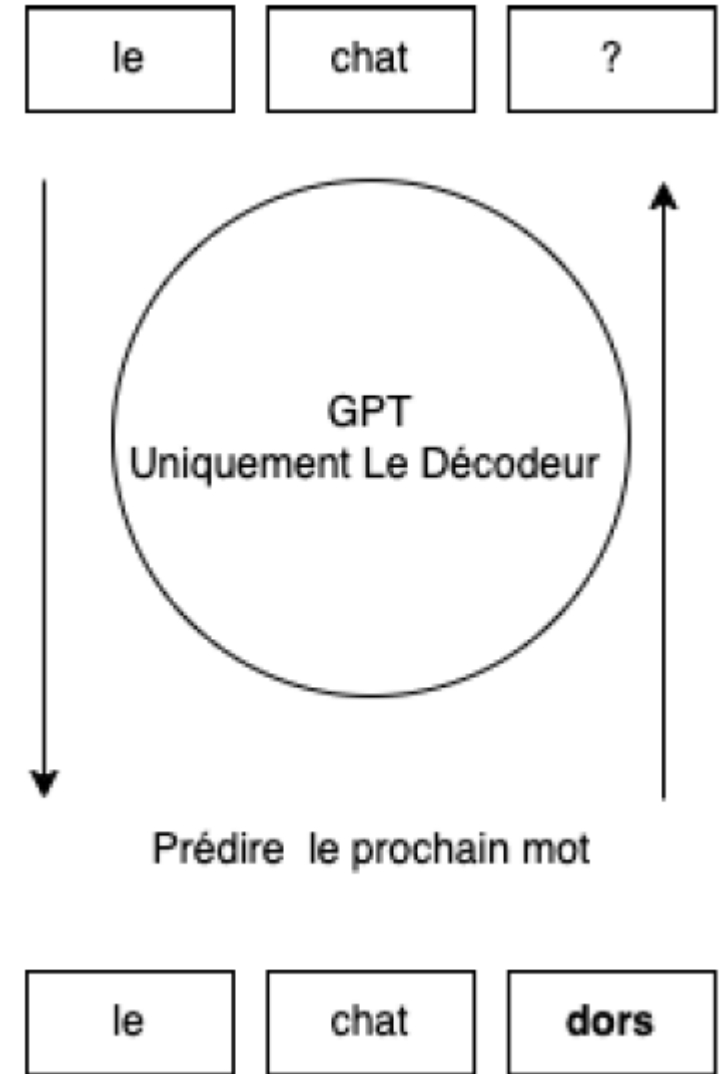
- RoBERTa
 - un **modèle développé à partir de BERT en modifiant certains hyperparamètres pendant la phase de pré-entraînement.**
- Contrairement à RoBERTa et FlauBERT, **CamemBERT**
 - **utilise l'algorithme SentencePiece pour la tokenisation et a recours à un modèle de masque du mot entier.**
 - Tout comme RoBERTa, les tokens sont masqués dynamiquement pour tout le corpus durant la phase de prétraitement.

Modèles génératifs

Modèles génératifs de grande taille

- reposent sur **l'architecture des transformers unidirectionnelle** (de gauche à droite), où **chaque token ne peut se référer qu'aux tokens qui le précèdent**, d'où leur appellation de modèles auto-régressifs
- divers modèles tels que ChatGPT, LLaMA 3, Mistral, Phi 3
- particulièrement utilisés pour des tâches telles que la traduction automatique, le résumé de texte, et la génération de réponses dans des systèmes de dialogue.

Modèle auto-régressif



Modèles génératifs de grande taille

- Certains de ces modèles ne sont pas open source et **offrent des API payantes**, comme ChatGPT et Claude.
- D'autres modèles sont **disponibles** en versions compactes qui peuvent être exécutées localement et affinées pour des tâches spécifiques.
 - Le processus de fine-tuning consiste à charger un modèle pré-entraîné sur un large corpus général, puis à l'entraîner sur un nouveau corpus adapté à la tâche ciblée, comme des dialogues ou des articles techniques.

Ingénierie des prompts : définition

- domaine clé pour **optimiser les performances des LLMs sans modifier leurs paramètres internes**.
- concevoir soigneusement l'entrée textuelle (instructions, contexte, exemples) pour guider le modèle vers la meilleure réponse possible.
- Stratégies :
 - **zero-shot/few-shot prompting** : fournir quelques exemples dans l'invite sans entraînement spécifique
 - **Chain-of-Thought** (chaîne de pensée) : amener le modèle à dérouler son raisonnement

Ingénierie des prompts : définition

- Un **prompt** est le texte d'entrée (ou l'instruction) que l'on fournit à un modèle de langage.
- **L'ingénierie des prompts :**
 - **comprendre** comment le modèle interprète les instructions ;
 - **formuler** le prompt de façon claire, structurée et contextualisée ;
 - **ajuster le niveau de détail** pour contrôler le ton, le style ou la profondeur de la réponse ;
 - **expérimenter et itérer** pour améliorer la performance du modèle sur une tâche donnée.

Principales stratégies d'ingénierie des prompts

- **Prompt contextuel (Contextual Prompting)**

- On fournit au modèle un contexte clair et complet : tâche, domaine, public cible, contraintes, format attendu.
 - Mauvais prompt : « Explique la grammaire. »
 - Bon prompt : « Explique la différence entre le passé composé et l'imparfait à un élève de 12 ans, avec des exemples simples. »

- **Prompt par rôle (Role Prompting)**

- On attribue un rôle ou une identité au modèle pour guider son ton ou son raisonnement.
 - « Tu es un professeur de linguistique. Explique le concept de complexité lexicale à des étudiants de master. »

Principales stratégies d'ingénierie des prompts

Zero-Shot Prompting

- On ne fournit **aucun exemple**, juste une instruction explicite.
 - Classe ces mots selon leur niveau de complexité lexicale (simple, moyen, complexe).

Few-Shot Prompting

- On montre **quelques exemples** de la tâche à accomplir (entrées + sorties) avant de donner un nouvel exemple à traiter.

- Few-shot learning prompt

prompt

*"Could you classify into « negative », « neutral » and « positive » the following sentence :
« the stock price slightly decreased by 1%? . Here are some examples : « SG stock fell by 1% : neutral »,
« Total stock increased by 1% : neutral », « BNP stock fell by 10% : negative ». Answer in one word*

Generation

"Neutral"

Principales stratégies d'ingénierie des prompts

- **Prompt avec raisonnement explicite (Chain-of-Thought Prompting)**

- On demande au modèle de raisonner étape par étape avant de donner la réponse finale.
 - Réfléchis étape par étape : d'abord identifie les indices linguistiques, puis explique comment ils influencent la compréhension.

prompt

"Count the number of "r" in the word "strawberry" . Proceede character by character to make the computation before answering"



Generation

I've counted 3 R's in the word "strawberry"

Principales stratégies d'ingénierie des prompts

- **Prompt de décomposition (Decomposition Prompting)**

- On divise une tâche complexe en sous-tâches successives, résolues étape par étape.
 - Identifier les mots complexes dans un texte.
 - Expliquer pourquoi ils sont complexes.
 - Proposer des simplifications.

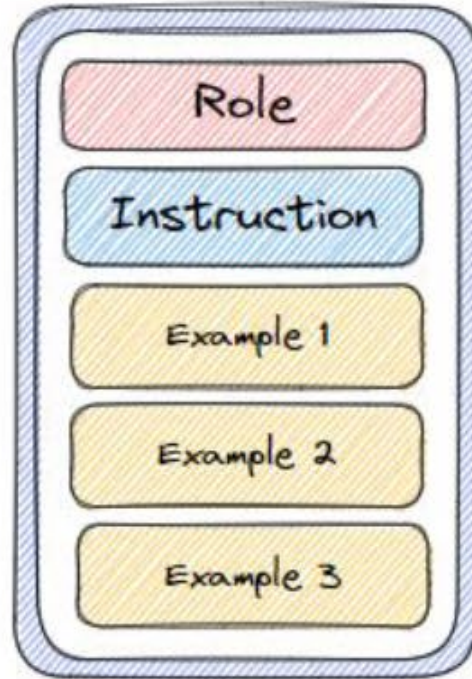
- **Prompt d'évaluation et d'amélioration**

- On demande au modèle d'auto-évaluer sa propre réponse ou d'en produire une version améliorée.
 - Relis ta réponse précédente et reformule-la de façon plus claire pour un public non expert.



Combining Techniques

A Combined Techniques Prompt



Twitter is a social media platform where users can post short messages called "tweets".

Tweets can be positive or negative, and we would like to be able to classify tweets as positive or negative. Here are some examples of positive and negative tweets. Make sure to classify the last tweet correctly.

Q: Tweet: "What a beautiful day!"
Is this tweet positive or negative?

A: positive

Q: Tweet: "I hate this class"
Is this tweet positive or negative?

A: negative

Q: Tweet: "I love pockets on jeans"

A:



Formalizing Prompts

You are a doctor.

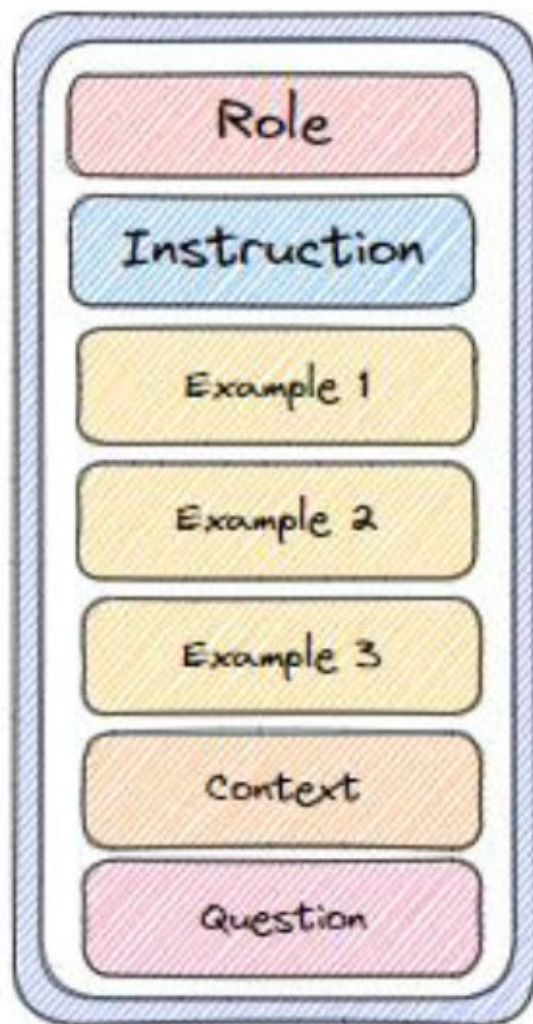
January 1, 2000: Fractured right arm playing basketball. Treated with a cast.

February 15, 2010: Diagnosed with hypertension. Prescribed lisinopril.

September 10, 2015: Developed pneumonia. Treated with antibiotics and recovered fully.

March 1, 2022: Sustained a concussion in a car accident. Admitted to the hospital and monitored for 24 hours.

Read this medical history and predict risks for the patient:



A good prompt structure

Interactive prompts



Instruction

a specific task or instruction you want the model to perform

Summarize, classify, define, how, what, etc.



Context

external information or additional context that can steer the model to better responses

e.g. I am a financial analyst



Input Data

the input or question that we are interested to find a response for

The article to summarize



Output Indicator

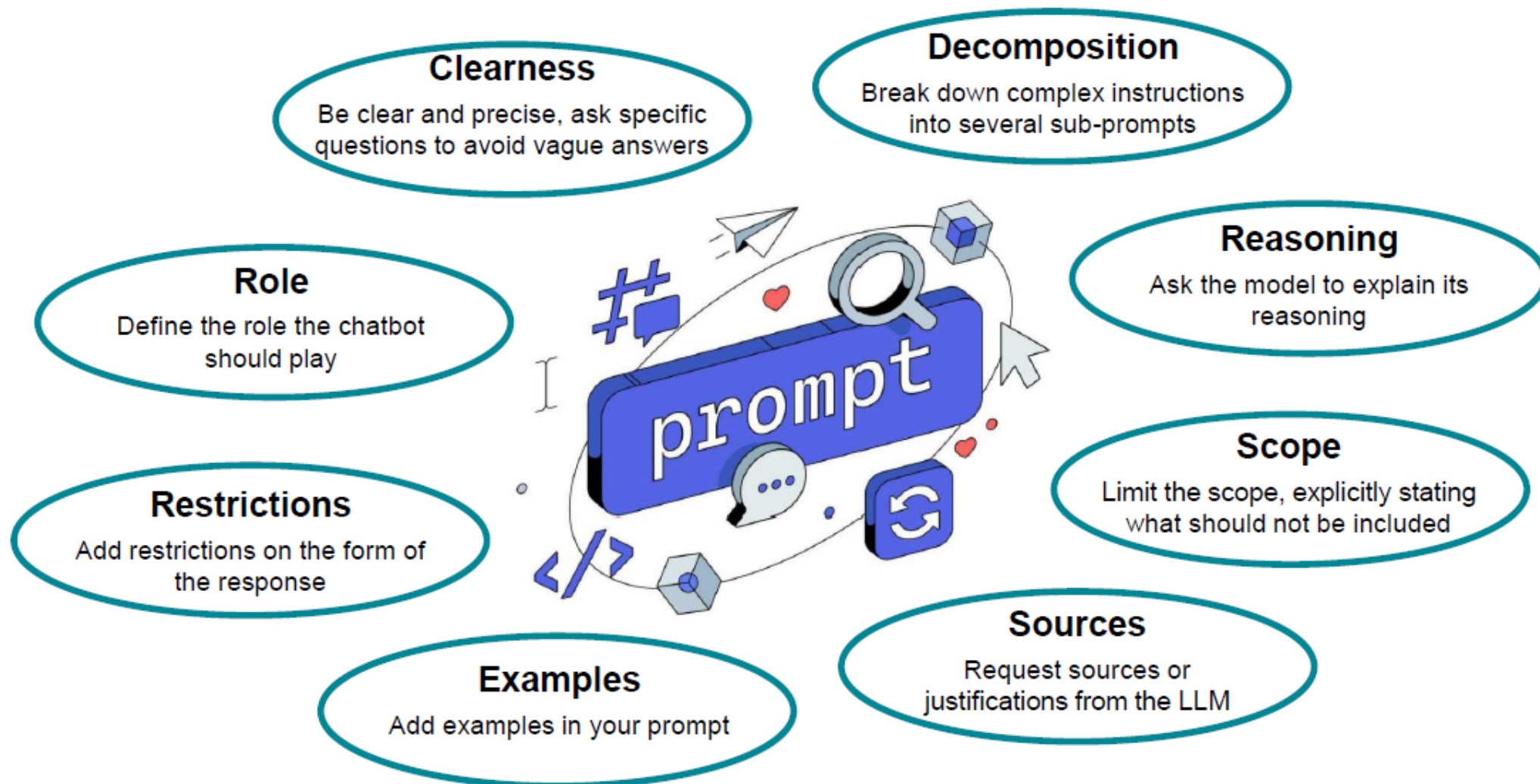
the type or format of the output.

The answer should be in one word, bullet points..

Prompting

How to efficiently use these models

The model answer depends heavily on your input text, also called prompt



Modèles génératifs de grande taille : problèmes

- **Limites de compréhension et de raisonnement**

- Les LLMs ne comprennent pas vraiment le sens comme un humain et prédisent la suite la plus probable d'un texte
- peuvent donc inventer des faits (on appelle ça des *hallucinations*).

- **Dépendance aux données d'entraînement**

- Ces modèles apprennent à partir de grandes quantités de textes existants.
- Si les données sont biaisées, incomplètes ou fausses, le modèle reproduit ces biais

- **Coût et impact environnemental**

- Entraîner un LLM demande des ressources énormes : puissance de calcul, électricité, mémoire, etc.
- Cela pose des questions d'empreinte carbone, d'accès inégal (seules quelques grandes entreprises peuvent le faire) et de durabilité.

Modèles génératifs de grande taille : problèmes

- **Problèmes éthiques et juridiques**

- Risque de désinformation, de plagiat, ou de violation du droit d'auteur
- Les modèles peuvent produire des contenus offensants ou discriminatoires s'ils ont appris à partir de textes contenant ces biais
- La transparence sur leur fonctionnement et leurs sources reste limitée.
- Surreprésentation de certaines langues => danger pour les langues peu dotées
- Surreprésentation d'un genre, d'une race, etc...

- **Difficulté d'interprétation**

- On ne sait pas toujours pourquoi un modèle donne une réponse donnée.
- Les mécanismes internes (comme l'attention dans les Transformers) sont difficiles à interpréter, donc c'est un peu une « boîte noire ».

- **Manque de créativité véritable et de bon sens**

- Les LLMs imitent la créativité, mais n'ont pas d'intentions ni de conscience du contexte réel.
- Ils peuvent produire des textes bien formulés mais vides de sens